Sonia Lerner
CSCI3212 HW2
9 November 2026

Note: ChatGPT formatted this document.

Problem 1: Hashing - from Keys to Slots

(Type: Asymptotic order, Hand trace, Analysis)

Figure 1. The keys `K = {12, 19, 26, 33, 5, 7}` are inserted, in that order, into a hash table with `M = 7` slots using `h(k) = k mod 7`.

1. **Direct addressing versus hashing.** The keys are 9-digit student IDs, so the key universe has `10^9` possible values, but only `n = 100` records are stored.

   - How much memory does direct addressing need, and what does a hash table with `M = 97` slots need?
     Direct addressing requires $O(M)$ memory. $M = 97$, so this hash table requires $O(97)$ memory.
   - Compute the load factor for `M = 97` and give the expected number of comparisons for a successful search with separate chaining under the simple uniform hashing assumption.
     Load factor $\alpha = \frac{n}{97}$
     Expected successful-search comparisons: $1 + \frac{\alpha}{2} = 1 + \frac{n}{194}$
   - Why does the lecture's advice say to pick `M` prime and not close to a power of 2 or 10?
     Picking a power of 2 or 10 will result in many items hashing to the same value, which will cause a lot of hash collisions.

2. **Separate chaining.** Insert the six keys of Figure 1 using separate chaining (new keys are appended to the end of their bucket).

   - Give `h(k)` for every key and draw the six buckets.
     Buckets: $0 : [7]$, $1 :$ empty, $2 :$ empty, $3 :$ empty, $4 :$ empty, $5 : [12, 19, 26, 33, 5]$, $6 :$ empty.
     Hashes: $h(12), h(19), h(26), h(33)$, and $h(5)$ all $= 5$, and $h(7) = 0$.
   - How many key comparisons does a lookup of `33` make? How many does a lookup of `8` make (a key that is not stored)?
     Lookup of 33: 4 comparisons $(12, 19, 26, 33)$. Lookup of 8: 0 comparisons because bucket $h(8) = 1$ is empty.
   - Five of the six keys land in the same bucket. Show why by writing the keys as `5 + 7t`, and explain what this says about the key set and about the choice of `M`.
     $5 = 5 + 7(0)$, $12 = 5 + 7(1)$, $19 = 5 + 7(2)$, $26 = 5 + 7(3)$, and $33 = 5 + 7(4)$. All five are congruent modulo 7 and therefore collide, making $M = 7$ a poor choice for these keys.

3. **Linear probing.** Now insert the same six keys, in the same order, into an open-addressing table with `M = 7` slots and linear probing (`h(k), h(k)+1, h(k)+2, ...` modulo 7); no resizing happens in this part. Count a *probe* for every slot examined, including the empty slot that ends a search.

   - Give the table after each insertion and the number of probes that insertion needed.
     After 12: [–,–,–,–,–,12,–] (1 probe)

After 19: [–,–,–,–,–,12,19] (2 probes)
After 26: [26,–,–,–,–,12,19] (3 probes)
After 33: [26,33,–,–,–,12,19] (4 probes)
After 5: [26,33,5,–,–,12,19] (5 probes)
After 7: [26,33,5,7,–,12,19] (4 probes).

- Give the final table, then count the probes for a lookup of `33` and for a failed lookup of `8`.
  Final table: [26,33,5,7,–,12,19].
  Lookup of 33 takes 4 probes (slots 5, 6, 0, 1), and the failed lookup of 8 takes 4 probes (slots 1, 2, 3, 4).
- Explain in two sentences why the key `5` needed five probes, and what *primary clustering* means here.
  Key 5 hashes to slot 5, but slots 5, 6, 0, and 1 are already occupied, so it takes five probes to reach slot 2. Primary clustering is the formation of long continuous runs of occupied slots, which cause later insertions and searches to require more probes.

4. **Load factor.**

- Compute the load factor after each of the six insertions. If the table doubles its capacity as soon as an insertion would push the load factor above `0.75`, after which insertion does the first resize happen, and what is the load factor immediately after that insertion?
  Load factors after each insertion: $1/7, 2/7, 3/7, 4/7, 5/7, 6/7$
  The first resize occurs on insertion 6 because $6/7 > 0.75$. After doubling to 14 slots, the load factor is $6/14 = 3/7$.
- Under uniform hashing, the expected probes for an unsuccessful search with open addressing is about `1/(1 - α)`. Give the values for α `= 0.5, 0.75, 0.9`, and say what this implies about resizing at `0.75` rather than at `0.95`.
  $\alpha = 0.5$: $\frac{1}{(1-\alpha)} = \frac{1}{0.5} = 2$ probes
  $\alpha = 0.75$: $\frac{1}{(1-\alpha)} = \frac{1}{0.25} = 4$ probes
  $\alpha = 0.9$: $\frac{1}{(1-\alpha)} = \frac{1}{0.1} = 10$ probes.
  Resizing at 0.75 keeps unsuccessful searches much cheaper than waiting until 0.95, since the estimate increases as α increases.
- Why can separate chaining tolerate α `> 1` while open addressing cannot?
  Separate chaining stores multiple keys in each bucket, so its chains can grow even when $n > M$. Open addressing stores each key in a distinct table slot, so it cannot hold more than M keys.

5. **Hashing strings.** A string `S` of length `L` is hashed with the polynomial `h(S) = (Σ s[i]·p^(L-1-i)) mod m`, where `p = 31` and `m = 97`, and `a = 1, b = 2, ..., z = 26`.

- Compute `h("cab")` and `h("acb")` (show the raw value before the modulus).
  $h(\text{"cab"}) = 3(31^2) + 1(31) + 2 = 2916 \to 2916 \bmod 97 = 6$.
  $h(\text{"acb"}) = 1(31^2) + 3(31) + 2 = 1056 \to 1056 \bmod 97 = 86$.
- Explain why the polynomial hash separates anagrams that a simple "sum of characters" hash would confuse.
  A sum-of-characters hash ignores character positions, so anagrams have the same sum. The polynomial hash assigns different powers of p to different positions, so rearranging characters usually changes the value.

- What properties should `p` and `m` have, and what is the risk of choosing `m = 2^16`?
  p should be larger than the alphabet size and m should be a large prime. With $m = 2^{16}$, we risk frequent clustering and collisions.

Problem 2: Tries and Prefix Queries

(Type: Hand trace, Analysis, Argument)

Figure 2. The dictionary `D = {cat, car, do, dog, dot}` is stored in a trie whose nodes have `children` (a map from characters to child nodes) and `is_end_of_word`.

1. **Build it.** Draw the trie. Mark every node with `is_end_of_word = True`. How many nodes does it have, including the root?
   It has 9 nodes including the root.

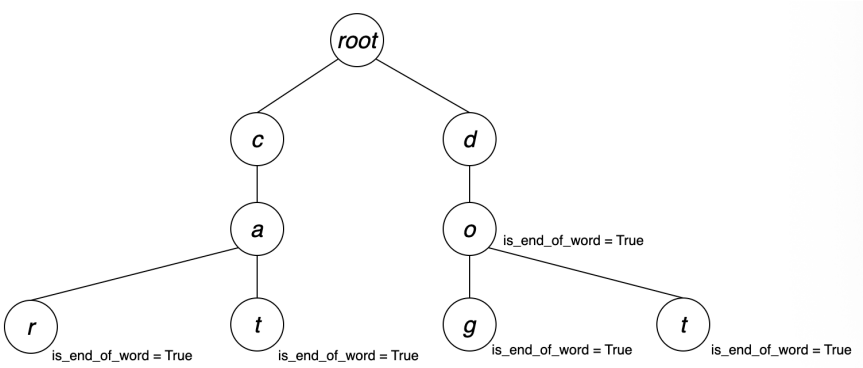


Figure 1: trie

2. **Lookups.** Trace `search("do")`, `search("ca")` and `search("can")`: how many nodes are visited, and why does the second one fail even though the path exists?
   search(do) visits root, *d*, *do* (3 nodes) and succeeds. *search*(ca) visits root, *c*, *ca* (3 nodes) but fails because *ca* is not marked as an end of a word. *search*(can) visits root, *c*, *ca* (3 nodes), then fails because there is no *n* child.

3. **Prefix queries.** List every word with prefix `do`. How many node steps does the query take (following the prefix plus enumerating the subtree)? How would a hash table answer the same query, and why is that expensive?
   The words are *do*, *dog*, and *dot*. The query follows 2 edges to the *do* node, then visits its 2 children to enumerate the subtree (4 edge steps, or 5 nodes including the root). A hash table has no prefix structure, so it must inspect all stored words (and test their prefixes), costing up to $O(NL)$.

4. **Cost.** Give the cost of insert, search, and delete for a key of length `L` in terms of `L`, and the space in terms of the number of words `N`, the maximum length `L`, and the alphabet size `|Σ|`. What changes if `children` is a hash map instead of an array?
   Access, insert, search, and delete each take $O(L)$ time. An array of children uses $O(NL)$ space in the worst case. A hash map uses expected $O(1)$ lookup and $O(NL)$ space.

5. **Connecting tries to string matching.** You are given a long text `T` (length `n`) and a dictionary `D`, and you must report every word of `D` that occurs in `T`.

   - Sketch a solution that walks the trie: at every start position of `T`, follow `T`'s characters through the trie and report words as they end. Give its running time in terms of `n`, the maximum word length `L`, and the size of the output, and say what it reports that a fixed-length window search does not.
     Start at each text position and follow trie edges while characters match, reporting every terminal node reached. The time is $O(nL + z)$, where z is the output size; unlike a fixed-length window search, it finds dictionary words of different lengths.
   - Sketch the alternative from the next problem: hash every length-`m` window of `T` and look the fingerprint up in a hash table of dictionary words. Give its expected time and the restriction it imposes on `D`.
     Preprocess the dictionary words of length m into a hash table of fingerprints, then roll a length-m window across the text and look up each fingerprint. Expected time is $O(km+n)$ including preprocessing (plus verification costs); this requires every dictionary word to have the same length m.
   - In two or three sentences, say which approach you would choose for (i) a static dictionary of 10,000 words of lengths 3 to 20 and a 100 MB text, and (ii) 1,000,000 words that all have length 20.
     For (i), use a trie because the dictionary contains different word lengths, so one traversal can report matches of lengths 3 through 20. For (ii), use rolling hashes and a hash table because all million words have the same length, allowing expected constant-time lookup per text window after preprocessing.

Problem 3: Rabin-Karp and Rolling Hashes

(Type: Hand trace, Derivation, Analysis)

Encoding from the lectures:

`a = 1, b = 2, ..., z = 26`, base `d = 10`, prime `q = 13`. The text is `T = "ababacab"` (length 8) and the pattern is `P = "aba"` (length 3); shifts are 0-based.

1. **Fingerprints.** Compute `value("aba")` and its fingerprint `p = value("aba") mod q`. Compute `h = d^(m-1) mod q` and the fingerprint `t_0` of the first window.
   $value(\text{aba}) = 1(10^2) + 2(10) + 1 = 121$, so $p = 121 \bmod 13 = 4$. Also, $h = 10^2 \bmod 13 = 9$, and the first window is aba, so $t_0 = 121 \bmod 13 = 4$.
2. **The scan.** Fill in a table with one row per shift `s = 0, ..., 5`, giving the window, its exact value, its fingerprint `t_s` (using the rolling update), whether `t_s` = `p`, and - for every hit - the result of verifying the characters. Which hit is spurious, and why does it not produce a wrong answer?
3. **Derivation.** Show that dropping `T[s+1]` and appending `T[s+m+1]` can be done in `O(1)`: `t_{s+1} = (d·(t_s - T[s+1]·h) + T[s+m+1]) mod q`, where `h = d^(m-1) mod q`. Explain why the subtraction is taken modulo `q` (that is, why the intermediate value may be negative).
   The outgoing character contributes $T[s+1]d^{m-1}$ to the old value; subtracting it removes the

leading digit. Multiplying the remainder by d shifts the remaining digits left, and adding the incoming character appends it. The subtraction may be negative, so taking the result modulo q maps it back to a valid fingerprint in $\{0, \ldots, q-1\}$.

4. **Complexity.** Give the preprocessing and matching costs, and the expected total. Then give an input on which Rabin-Karp degrades to the brute-force bound Θ`((n-m+1)·m)`. What does a large prime `q` change, and what does it not change?
   Preprocessing costs $O(m)$ and rolling through the text costs $O(n-m+1)$ expected time, so the expected total is $O(n+m)$ (with verification costs for fingerprint hits). The worst case is $T = a^n$ and $P =$ ana for $m = 3$: every window hashes like aaa modulo 13, so each shift requires $\Theta(m)$ verification and the total is $\Theta((n-m+1)m)$. A larger prime q reduces collision likelihood but does not eliminate collisions or improve the worst-case bound.

5. **Many patterns.** Explain how to search for `k` patterns at once using a hash table of fingerprints. What is the expected running time, what happens on a hit, and why is this useful for plagiarism detection (as in the lecture)?
   Compute and store the fingerprint of each of the k patterns in a hash table, mapping each fingerprint to its pattern(s). Roll the text window once and look up each window fingerprint in expected $O(1)$ time; on a hit, verify the characters to rule out collisions. This efficiently finds shared substrings or matching passages across many documents, which is useful for plagiarism detection.

Problem 4: Knuth-Morris-Pratt

(Type: Hand trace, Argument)

Figure 3. The pattern `P = "abababaca"` (length 7) and the text `T = "abababacaba"` (length 11). Positions are 1-indexed, and `q` denotes the number of characters of `P` matched so far.

1. **Prefix function.** Compute π`[i]` for `i = 1, ..., 7` and fill in the table. State the definition you used. Show how π`[5] = 3` is obtained from `P[1..5] = "ababa"`.
   The prefix function is the length of the longest proper prefix that is also a suffix: $\pi = [0, 0, 1, 2, 3, 0, 1]$. For "ababa", the longest proper prefix that is also a suffix is "aba", of length 3, so $\pi[5] = 3$.

2. **Trace.** Run the KMP matcher on `T`, giving a table with columns: step, `i`, `T[i]`, `q` before, the action (`extend`, or each fallback `q` ← π`[q]`), and `q` after. Mark the mismatch that forces a fallback and the position where a match is reported. How many character comparisons does the trace make in total (a fallback is not a comparison)?
   Trace: $i = 1$–5 extend $q : 0 \to 1 \to 2 \to 3 \to 4 \to 5$. At $i = 6$ (b), mismatch with P[6]=c; fall back $q = \pi[5] = 3$, then b matches P[4], so $q = 4$. At $i = 7$ (a), $q = 5$; at $i = 8$ (c), $q = 6$; at $i = 9$ (a), $q = 7$ and report a match starting at position 3. After the match, set $q = \pi[7] = 1$; $i = 10$ (b) gives $q = 2$, and $i = 11$ (a) gives $q = 3$. Total: 12 character comparisons.

3. **Why it is linear.** Argue that the total number of comparisons is `O(n + m)`: say why `q` increases by at most one per text character, why every fallback strictly decreases `q`, and why π`[q] < q` for `q >= 1`.
   q increases by at most one for each text character, so its total increases are $O(n)$. Every fallback strictly decreases q, and $\pi[q] < q$ because a proper prefix is shorter than the string;

therefore the total number of fallbacks is bounded by the total increases plus the initial q, giving $O(n)$ comparisons. Including the $O(m)$ prefix-function preprocessing gives $O(n+m)$.

4. **Worst case for the naive matcher.** Take `T = a^n` and `P = a^(m-1)b`. Count the comparisons the naive matcher makes and the comparisons KMP makes. (Use `n = 20, m = 5` for the arithmetic, then give the general bounds.)
   For $n = 20, m = 5$, the naive matcher checks 5 characters at each of $20 - 5 + 1 = 16$ shifts: $16 \cdot 5 = 80$ comparisons. KMP makes 4 initial matches, then for each of the remaining 16 characters it makes one mismatch comparison and one comparison after fallback: $4 + 16 \cdot 2 = 36$. In general, the naive bound is $\Theta((n-m+1)m)$, while KMP is $O(n+m)$.

5. **A second pattern.** Compute the prefix function of `P' = "aabaab"`, and say where KMP would fall back while matching `P'` against `T' = "aabaabaab"`.
   π for "aabaab" is $[0, 1, 0, 1, 2, 3]$. The first occurrence matches at positions 1–6; after reporting it, KMP falls back to $q = \pi[6] = 3$ to preserve the overlapping suffix "aab". It then matches positions 7–9 and reports the second occurrence at positions 4–9.

Problem 5: Approximate Matching with One Mismatch

(Type: Hand trace, Argument, Analysis)

The problem. The alphabet is Σ `= {a, b, c, d}`, the pattern length is `m = 3`, the dictionary is `D = {abc, abd, acd, bad, cab}`, and the text is `T = "abdcabb"` (length 7). Every word of `D` has length `m`.

- The **Hamming distance** `H(x, y)` of two strings of length `m` is the number of positions where they differ (substitutions only; it is undefined for different lengths).
- A word `w` has an **approximate occurrence at shift** `s` if `H(T[s+1..s+m], w) <= 1`. Exact occurrences count as approximate ones.
- The output is every pair `(s, w)` with an approximate occurrence, sorted by `s` and then by `w`.

1. **Warm-up.** Fill in a table of `H(window, w)` for every shift and every `w` $\in$ `D`, and list the output.

2. **Baseline.** Describe the brute-force algorithm and give its running time in terms of `n`, `m`, and the dictionary size `k`. How does it relate to the naive matcher's bound from the lectures?
   For each of the $n - m + 1$ windows, compare it with all k dictionary words, checking up to m characters per comparison. The running time is $O((n-m+1)km)$, or $O(nkm)$. This is the naive matching bound $O((n-m+1)m)$ repeated for each of the k words.

3. **Trie search with a mismatch budget.** Insert `D` into a trie and run the following search on one window `W`:

```
APPROX_SEARCH(v, W, b)              // v is a trie node, j = depth(v)
    j = depth(v)
    if j == m:
        if v.is_end_of_word:
            report string(v)
```

```
        return
    for each (c, child) in v.children, in alphabetical order of c:
        if c == W[j+1]:
            APPROX_SEARCH(child, W, b)
        else if b > 0:
            APPROX_SEARCH(child, W, b {-}
```

The search starts with `APPROX_SEARCH(root, W, 1)`.

- Draw the trie for `D` and count its nodes.
  root has children a, b, c. The paths are $a \to b \to c$ (abc), $a \to b \to d$ (abd), $a \to c \to d$ (acd), $b \to a \to d$ (bad), and $c \to a \to b$ (cab). Mark all five depth-3 nodes as word endings. There are $1 + 3 + 4 + 5 = \boxed{13}$ nodes, including the root.
- Trace the search on the window at shift `s = 4` (`W = "abb"`): list the nodes visited, in order, with the remaining budget, and the words reported.
  `root(1)` → `a(1)` → `ab(1)` → `abc(0)` → `abd(0)` → `ac(0)` → `b(0)` → `c(0)`. The reported words are `abc` and `abd`.
- Explain why `b = 0` behaves exactly like an ordinary trie lookup.
  With no mismatches remaining, the algorithm follows only the child matching the next window character. It cannot explore alternative characters, so it behaves like an exact trie lookup.

4. **Correctness.** Prove that the search reports exactly the words `w` ∈ `D` with `H(W, w) <= 1`. (Relate the budget `b` to the number of mismatches between `W[1..j]` and `string(v)`.)
   At depth j, the remaining budget is $b = 1 - r$, where r is the number of mismatches between the path label and the first j characters of W. A matching edge leaves the budget unchanged; a nonmatching edge is followed only if $b > 0$ and decreases it by one. Thus every path with at most one mismatch is explored, while paths with more than one mismatch are pruned. At depth m, exactly the dictionary words at Hamming distance at most one are reported.

5. **Cost.** Show that one call visits at most `1 + m + (|Σ| - 1)*m(m+1)/2` nodes, and conclude that the whole algorithm runs in `O((n - m + 1)·|Σ|·m²)` time, independent of `k`. When is that better than the baseline of part 2?
   There is one root and at most m nodes on the exact-match path. At each of the m positions, at most $|\Sigma| - 1$ mismatch branches can be explored, each extending for at most the remaining number of positions. Their total is $(|\Sigma| - 1)\sum_{i=1}^{m}(m - i + 1) = (|\Sigma| - 1)m(m + 1)/2$. Thus one call has the stated bound, and all windows take $O((n - m + 1)|\Sigma|m^2)$. This is better than the baseline when roughly $|\Sigma|m < k$.

6. **A hashing alternative.** Instead of a trie, store the fingerprints of all dictionary words (with the encoding `a = 1, ..., d = 4`, base `d = 10`, prime `q = 13`). For a window with fingerprint `t`:

   - show that changing the character at position `j` from `c` to `c'` gives the fingerprint `t' = (t + (c' - c)·d^(m-j)) mod q`, which is `O(1)` with precomputed powers;
     Only the term for position j changes, from $c\,d^{m-j}$ to $c'\,d^{m-j}$. Therefore $t' = (t + (c' - c)d^{m-j}) \bmod q$, computable in $O(1)$ with precomputed powers.
   - describe an algorithm that rolls the window and, at each shift, looks up the window's fingerprint and the fingerprints of its `m(|Σ| - 1)` one-substitution variants in a hash set, verifying characters on every hit;

For each window, compute its fingerprint and generate the fingerprints for all $m(|\Sigma|-1)$ possible one-character substitutions using the formula above. Look up the original and variant fingerprints in the dictionary hash set; verify the actual strings on every hit to rule out collisions. Update the window fingerprint in $O(1)$ when shifting to the next window.

- give its expected running time and compare its time and space with the trie and with the variant that stores the fingerprints of all one-substitution variants of the dictionary words.
  Querying all variants takes expected $O((n-m+1)m|\Sigma|)$ time, plus character verification for hits, and $O(k)$ space for the dictionary fingerprints. Precomputing and storing every dictionary word's one-substitution variants takes $O(km|\Sigma|)$ space but reduces searching to expected $O(n-m+1)$ hash lookups, plus verification. The trie takes $O((n-m+1)|\Sigma|m^2)$ time and $O(km)$ space. Since $q = 13$ is small, collisions may cause extra verification work.

7. **Why not KMP?** In two or three sentences, explain why the prefix function π cannot be reused directly when one mismatch is allowed.
   KMP's prefix function relies on exact matches to identify reusable prefix/suffix overlaps. With one mismatch allowed, partial matches may contain substitutions, so the exact overlap information in π does not represent all possible states. The algorithm would need to track both the matched position and the remaining mismatch budget.

Problem 6: Graph Representations - Adjacency Matrices versus Adjacency Lists

(Type: Asymptotic order, Hand trace)

Figure 4. An undirected graph on vertices `1, 2, 3, 4, 5, 6` with the edges `{1,2}, {1,3}, {2,4}, {3,4}, {4,5}, {5,6}`.

1. **Write both representations.** Give the `6 × 6` adjacency matrix and the adjacency lists of Figure 4. How many entries of the matrix are non-zero, and what fraction of the matrix is that?

$$\begin{pmatrix} 0 & 1 & 1 & 0 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 1 & 0 & 0 & 1 & 0 & 0 \\ 0 & 1 & 1 & 0 & 1 & 0 \\ 0 & 0 & 0 & 1 & 0 & 1 \\ 0 & 0 & 0 & 0 & 1 & 0 \end{pmatrix}$$

   Adjacency lists: $1 : \{2,3\}$; $2 : \{1,4\}$; $3 : \{1,4\}$; $4 : \{2,3,5\}$; $5 : \{4,6\}$; $6 : \{5\}$. There are $2E = 12$ nonzero entries out of 36, so the fraction is $\boxed{1/3}$.

2. **Storage.** For a graph with `V = 6 000` vertices and `E = 15 000` edges, how many cells does the adjacency matrix store (and how many megabytes is that at 4 bytes per cell), and how many entries does the adjacency list store (at 8 bytes per entry)? Compute the ratio.
   The matrix stores $V^2 = 6000^2 = 36{,}000{,}000$ cells, using 144,000,000 bytes = 144 MB. The adjacency list stores $2E = 30{,}000$ entries, using 240,000 bytes = 0.24 MB. The matrix uses $\boxed{600}$ times as much storage.

3. **Operations.** Which representation answers "are `u` and `v` adjacent?" in `O(1)`, and which one lists all neighbours of `v` in `O(deg(v))`? What does one sweep that examines every edge cost in each representation?
   The adjacency matrix tests adjacency in $O(1)$. The adjacency list lists all neighbours of v in $O(\deg(v))$. A full sweep costs $O(V^2)$ with a matrix and $O(V + E)$ with adjacency lists.

4. **Choosing.** In which situations is the adjacency matrix the better choice? Give one concrete example and justify it in two or three sentences.
   An adjacency matrix is preferable for dense graphs or when adjacency queries are frequent. For example, in a small, nearly fully connected network, checking whether any two vertices share an edge takes $O(1)$, and the matrix's storage cost is manageable.